

SOFTWARE MAINTENANCE

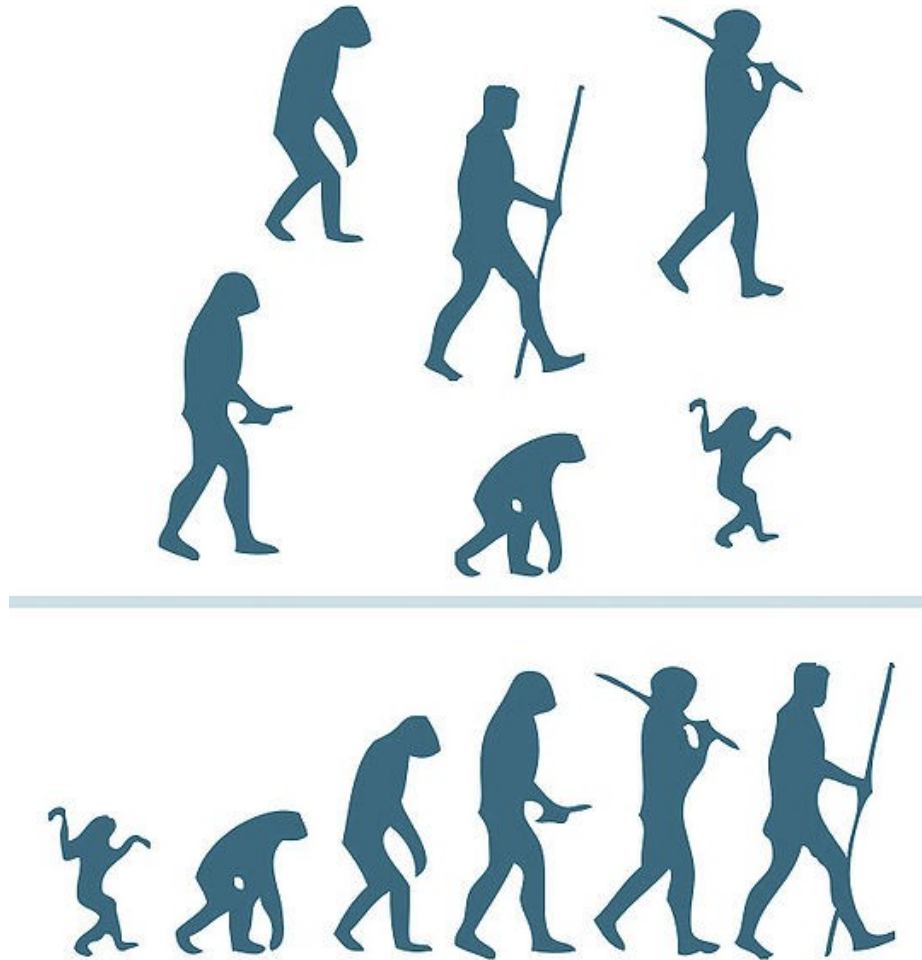


MANUTENÇÃO DE SOFTWARE

Prof. Raul Sidnei Wazlawick

UFSC-CTC-INE

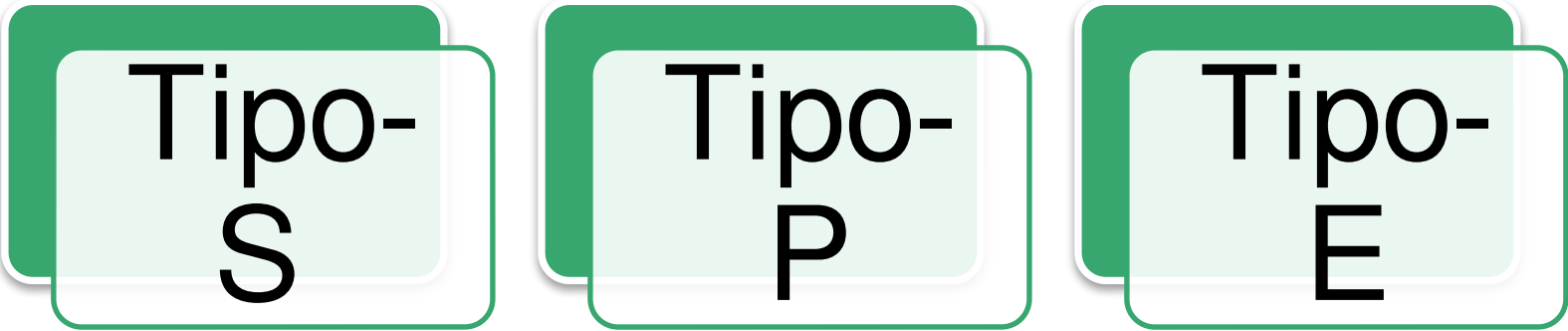
MANUTENÇÃO X EVOLUÇÃO DE SOFTWARE



LEIS DE LEHMAN



TIPOS DE SISTEMA:



Tipo-
S

Tipo-
P

Tipo-
E

1. LEI DA MUDANÇA CONTÍNUA

- Um sistema que é efetivamente usado deve ser **continuamente melhorado**, caso contrário torna-se cada vez menos útil, pois seu contexto de uso evolui.



2. LEI DA COMPLEXIDADE CRESCENTE

- À medida que um programa evolui, sua complexidade inerente aumenta, porque as correções feitas podem deteriorar sua organização interna.



3. LEI FUNDAMENTAL DA EVOLUÇÃO DE PROGRAMAS: AUTO REGULAÇÃO

- A evolução de programas é sujeita a uma dinâmica de autorregulação que faz com que medidas globais de esforço e outros atributos de processo sejam estatisticamente previsíveis (distribuição normal).



4. LEI DA CONSERVAÇÃO DA ESTABILIDADE ORGANIZACIONAL: TAXA DE TRABALHO INVARIANTE

- A taxa média efetiva de trabalho global em um sistema em evolução é invariante no tempo: ela não aumenta nem diminui.



5. LEI DA CONSERVAÇÃO DA FAMILIARIDADE: COMPLEXIDADE PERCEBIDA

- Durante a vida ativa de um programa, o conteúdo das sucessivas versões do programa (mudanças, adições e remoções) é estatisticamente invariante.



6. LEI DO CRESCIMENTO CONTÍNUO

- O conteúdo funcional de um sistema deve crescer continuamente para manter a satisfação do usuário.



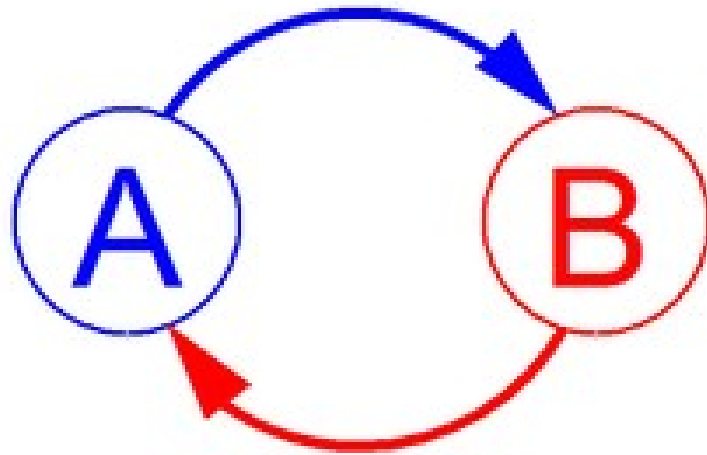
7. LEI DA QUALIDADE DECRESCENTE

- A qualidade de um sistema vai parecer diminuir com o tempo, a não ser que medidas rigorosas sejam tomadas para manter e adaptar o sistema.

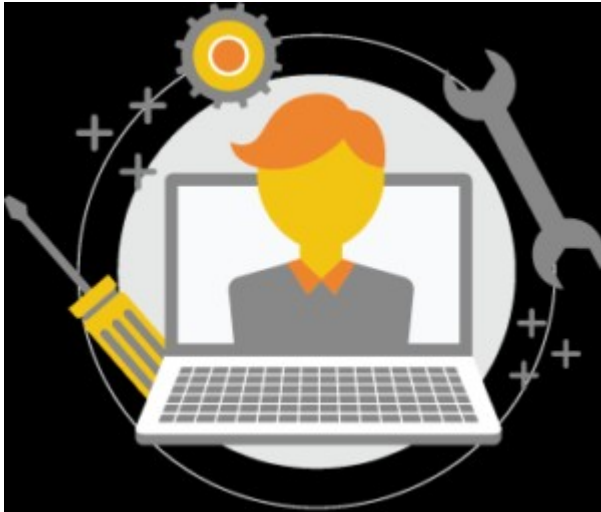


8. LEI DO SISTEMA REALIMENTADO

- A evolução de sistemas é um processo multi-nível, multi-*loop* e multi-agente de realimentação, e deve ser encarado dessa forma para que se obtenha melhorias significativas em uma base razoável.



CLASSIFICAÇÃO DAS ATIVIDADES DE MANUTENÇÃO



Corretiva

Adaptativa

Perfectiva

Preventiva



MANUTENÇÃO CORRETIVA



Manutenção para
correção de erros
conhecidos.



Manutenção para
detecção e correção de
novos erros.



MANUTENÇÃO ADAPTATIVA

Requisitos de cliente e usuário mudam com o passar do tempo.

Novos requisitos surgem.

Leis e normas mudam.

Tecnologias novas entram em uso.

Etc.

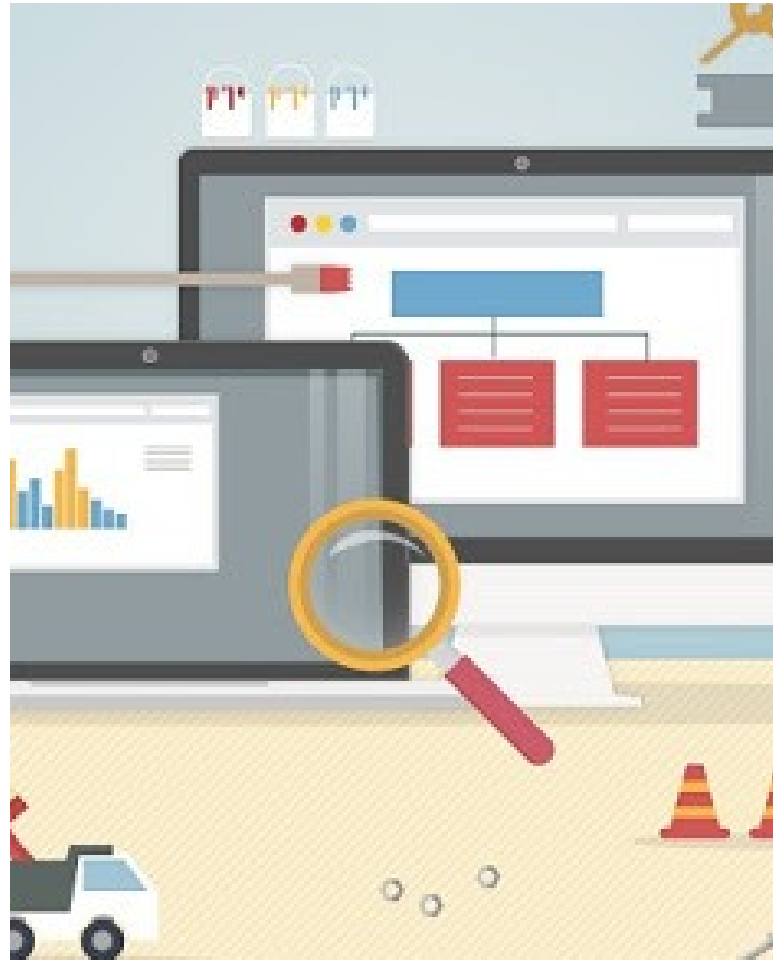


REQUISITOS PERMANENTES E TRANSITÓRIOS



MANUTENÇÃO PERFECTIVA

- Consiste em mudanças que afetam mais as características de desempenho do que de funcionalidade do software.



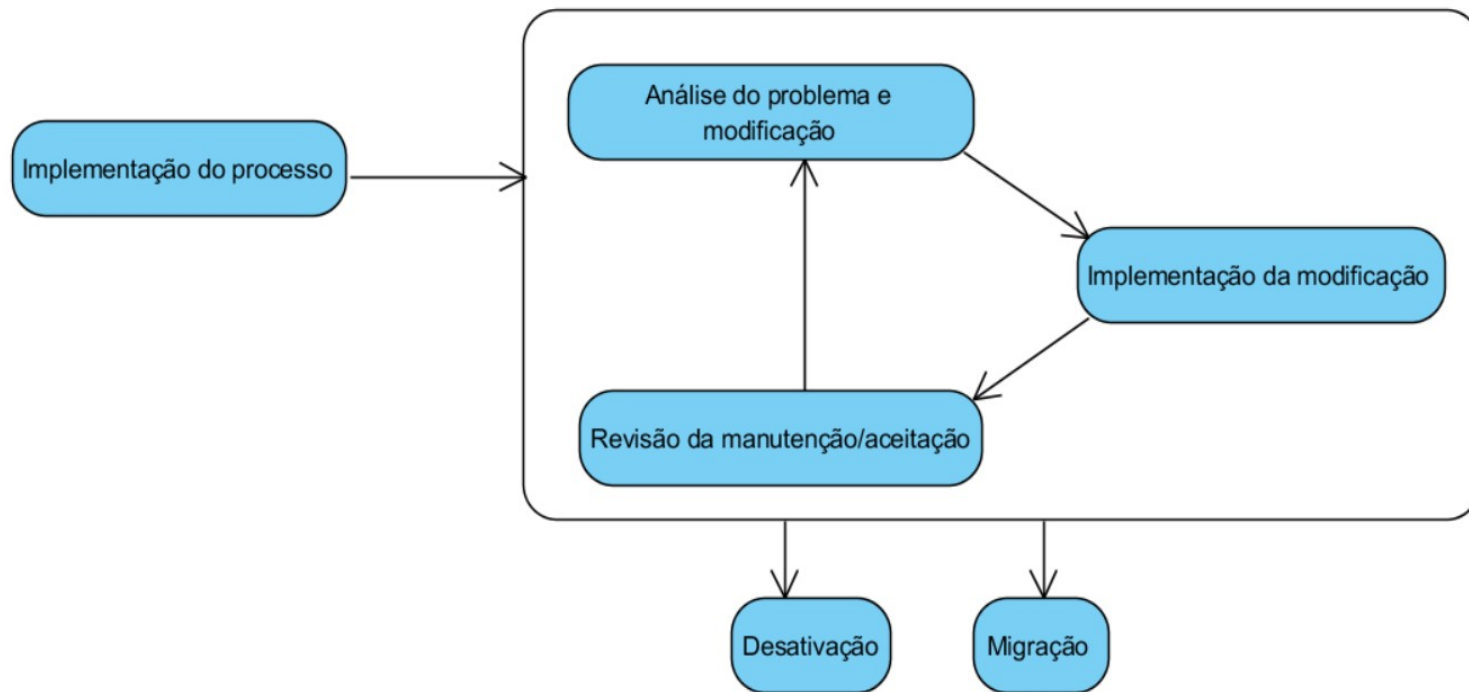
MANUTENÇÃO PREVENTIVA



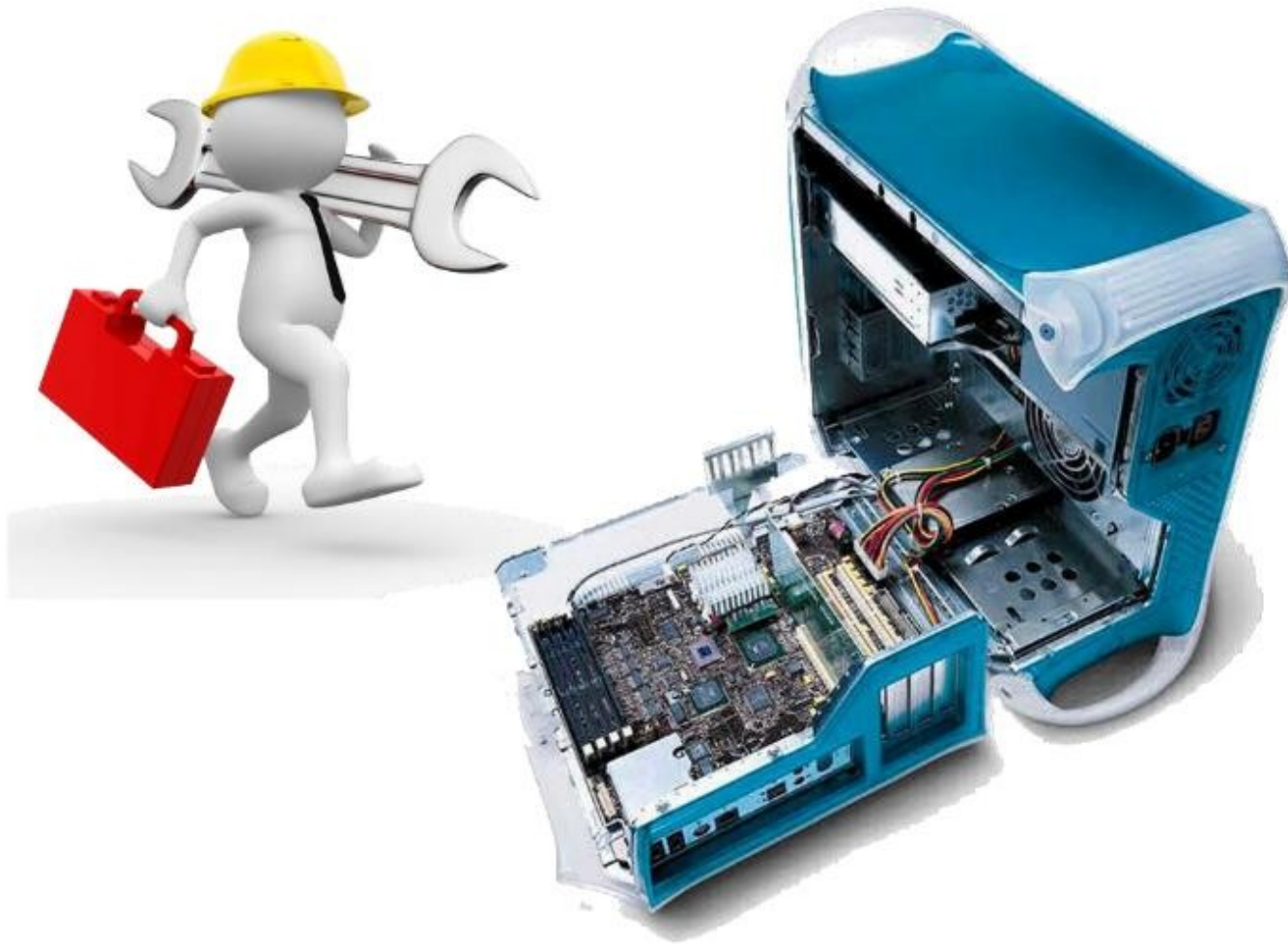
- Pode ser realizada através de atividades de reengenharia, nas quais o software é modificado para resolver problemas potenciais.



PROCESSO DE MANUTENÇÃO



TIPOS DE ATIVIDADES DE MANUTENÇÃO



REPARAÇÃO DE DEFEITOS



Tabela 14-1: Tempo de Resposta ao Erro em Função de sua Severidade¹⁹⁸.

Severidade	Significado	Tempo nominal da descoberta aos reparos iniciais	Percentual em relação aos defeitos relatados
1	Aplicação não funciona	1 dia	1%
2	Funcionalidade principal não funciona	2 dias	12%
3	Funcionalidade secundária não funciona	30 dias	52%
4	Erro cosmético	120 dias	35%

FATORES QUE PODEM INFLUENCIAR A ESTIMAÇÃO DE ESFORÇO A SER APLICADA ÀS ATIVIDADES DE REPARAÇÃO DE DEFEITOS:

- *Defeitos suspensos (abeyant).*
- *Defeitos inválidos.*
- *Consertos ruins (bad fix injection).*
- *Defeitos duplicados.*



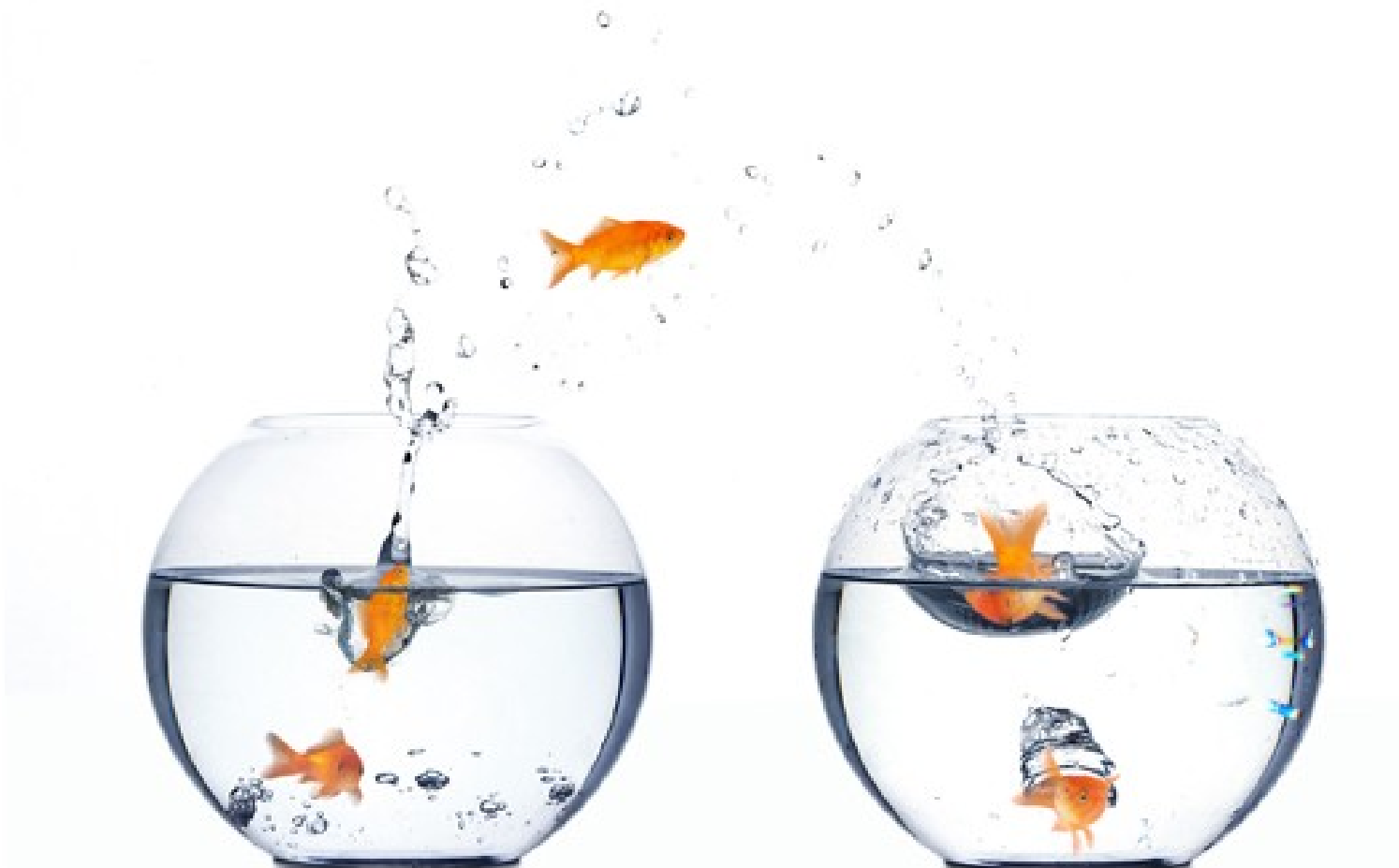
REMOÇÃO DE MÓDULOS SUJEITOS A ERROS



SUPORE A USUÁRIOS



MIGRAÇÃO ENTRE PLATAFORMAS



CONVERSÃO DE ARQUITETURA



ADAPTAÇÕES OBRIGATÓRIAS



OTIMIZAÇÃO DE PERFORMANCE



MELHORIAS



- *Pequenas melhorias,*
 - consistindo de aproximadamente 5 pontos de função, ou seja, a introdução de um novo relatório, consulta ou tela.
- *Grandes melhorias,*
 - consistindo de um número significativamente maior de pontos de função, tipicamente mais de 20 pontos de função, que devem ser tratadas como pequenos projetos de desenvolvimento.

MODELOS DE ESTIMAÇÃO DE ESFORÇO DE MANUTENÇÃO



ACT



COCOMO II-
MANUTENÇÃO
O



FP E SMPEEM



MODELO ACT

- O *modelo ACT* baseia-se em uma estimativa da percentagem de linhas de código que vão sofrer manutenção.
- São consideradas linhas em manutenção tanto as linhas de código novas criadas quanto as linhas alteradas durante a manutenção.
- O valor da variável ACT é então número de linhas que sofrem manutenção dividido pelo número total de linhas do código em um ano típico.

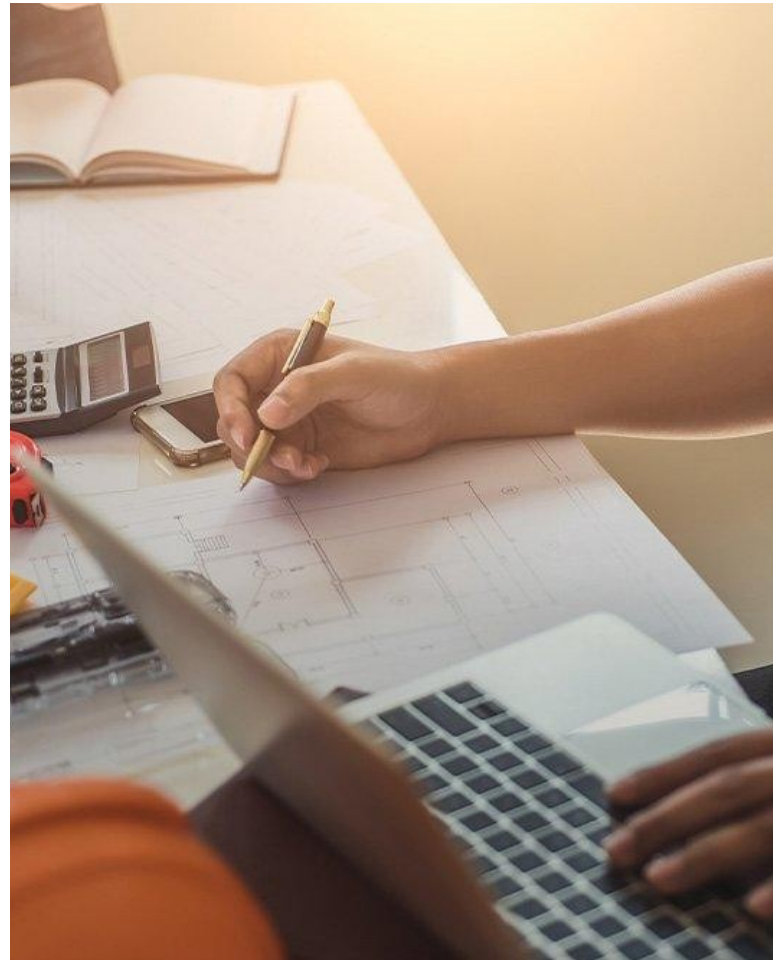


ACT

- Esforço estimado de manutenção durante um ano:

$$E = ACT * SDT$$

- ACT = porcentagem de linhas a sofrer manutenção.
- SDT = Software Development Time



EXEMPLO

- Um software que foi desenvolvido com um esforço de 80 desenvolvedor-mês terá $SDT = 80$.
- Se a taxa anual esperada de linhas em manutenção (ACT) for de 2%, então o esforço anual esperado de manutenção para este software será dado por:
- $E = 0,02 * 80 = 1,6$

VARIAÇÃO DE SCHAEFER

- $E = ACT * 2,4 * KSLOC^{1,05}$

EXEMPLO

- Assim, um software com 20 mil linhas de código e ACT de 2% teria o seguinte esforço anual de manutenção (em desenvolvedor-mês):
- $E = 0,02 * 2,4 * 20^{1,05} = 1,115$

MODELO DE MANUTENÇÃO DE CII

A equação de estimação de esforço para manutenção é semelhante à equação usada no modelo *post-architecture*, com a seguinte forma:

$$E = A * KSLOC_m^S * \prod_{i=1}^n M_i$$

Onde:

- E é o esforço de manutenção em desenvolvedor-mês a ser calculado.
- A é uma constante calibrada pelo método, inicialmente valendo 2,94.
- $KSLOC_m$ é o número de linhas de código que se espera adicionar ou alterar ajustadas pelo fator de manutenção (ver abaixo). Não são contadas as linhas que eventualmente serão excluídas do código.
- S é o coeficiente de esforço, determinado pelos fatores de escala, e calculado como mostrado na Seção 7.3.
- M_i são os multiplicadores de esforço.

MUDANÇAS EM RELAÇÃO AO MODELO *POST-ARCHITECTURE* PARA O CÁLCULO DO ESFORÇO DE MANUTENÇÃO:

- **SCED**

(Cronograma de Desenvolvimento Requerido) não é usado porque se espera que ciclos de manutenção tenham duração fixa predeterminada.

- **RUSE**

(Desenvolvimento para Reuso) não é usado porque se considera que o esforço requerido para manter a reusabilidade de um componente de software é balanceada pela redução do esforço de manutenção devido ao projeto, documentação e teste cuidadosos do componente.

- **RELY**

(Software com Confiabilidade Requerida) tem uma tabela de aplicação diferenciada.

Tabela 14-2: Forma de obtenção do equivalente numérico para RELY na fase de manutenção.

Descritor	Pequena inconveniência	Perdas pequenas, facilmente recuperáveis	Perdas moderadas, facilmente recuperáveis	Alta perda financeira	Risco a vida humana	
Avaliação	Muito baixo	Baixo	Nominal	Alto	Muito Alto	Extra-alto
Equivalente numérico	1,23	1,10	1,00	0,99	1,07	n/a

O número de *KSLOC* usado na fase de manutenção deve ser ajustado antes de ser aplicado na equação de cálculo de esforço pelo uso do *fator de ajuste de manutenção (MAF)*:

$$KSLOC_m = (KSLOC_{adicionadas} + KSLOC_{modificadas}) * MAF$$

O fator de ajuste de manutenção *MAF* é calculado a partir da equação a seguir:

$$MAF = 1 + \left(\frac{SU}{100} * UNFM \right)$$

Onde:

- a) *SU* é o fator de ajuste relacionado à *compreensão do software (software understanding)*, calculado de acordo com a Tabela 14-3.
- b) *UNFM* é o fator de *não familiaridade* com relação ao software, calculado de acordo com a Tabela 14-4.

Tabela 14-3: Forma de cálculo do equivalente numérico para *SU*.

	Muito baixo	Baixo	Nominal	Alto	Muito alto
Estrutura	Coesão muito baixa, acoplamento alto, código espagete	Coesão moderadamente baixa, acoplamento alto	Razoavelmente bem estruturado, algumas áreas fracas	Alta coesão, baixo acoplamento	Modularidade forte, ocultamento de informação em estruturas de dados ou controle (objetos)
Clareza da aplicação	As visões do programa e sua aplicação no mundo real não batem	Alguma correlação entre o programa e a aplicação	Correlação moderada entre o programa e a aplicação	Boa correlação entre o programa e a aplicação	As visões do programa e da aplicação no mundo real claramente batem
Auto-descrição	Código obscuro, documentação faltando, obscura ou obsoleta	Alguns comentários no código e cabeçalhos, alguma documentação útil	Nível moderado de documentação no código e cabeçalhos	Código e cabeçalhos bem documentados, algumas áreas fracas	Código autodescritivo, documentação atualizada e bem organizada baseada em <i>design</i>
Valor de <i>SU</i>	50	40	30	20	10

Tabela 14-4: Forma de cálculo do equivalente numérico para *UNFM*.

Nível de não familiaridade	Valor de <i>UNFM</i>
Completamente familiar	0,0
Basicamente familiar	0,2
Um tanto familiar	0,4
Um tanto não familiar	0,6
Basicamente não familiar	0,8
Completamente não familiar	1,0

MODELOS FP E SMPEEM

- Segundo este modelo é necessário calcular os pontos de função não ajustados de quatro tipos de funções:
 - *ADD*: UFP de funções que vão ser adicionadas.
 - *CHG*: UFP de funções que vão ser alteradas.
 - *DEL*: UFP de funções que vão ser removidas.
 - *CFP*: UFP de funções que serão adicionadas por conversão.



- Além de classificar as entradas, saídas, consultas, arquivos internos e arquivos externos nestes quatro tipos antes de contabilizar seus pontos de função não ajustados, a técnica propõe que os fatores de ajuste técnico (VAF) sejam calculados para dois momentos:
 - antes da manutenção VAF_A e
 - depois da manutenção VAF_D .
- A equação seguinte é então aplicada:

$$E = (ADD + CHG + CPF) * VAF_D + DEL * VAF_A$$

EVOLUÇÃO SMPEEM

- Inclui mais 10 fatores de ajuste específicos para as atividades de manutenção:
 - Conhecimento do domínio da aplicação.
 - Familiaridade com a linguagem de programação.
 - Experiência com o software básico (sistema operacional, gerenciador de banco de dados).
 - Estruturação dos módulos de software.
 - Independência entre os módulos de software.
 - Legibilidade e modificabilidade da linguagem de programação.
 - Reusabilidade de módulos de software legados.
 - Atualização da documentação.
 - Conformidade com padrões de engenharia de software
 - Testabilidade.



ENGENHARIA REVERSA E REENGENHARIA



Taxionomia de Chikofsky e Cross II

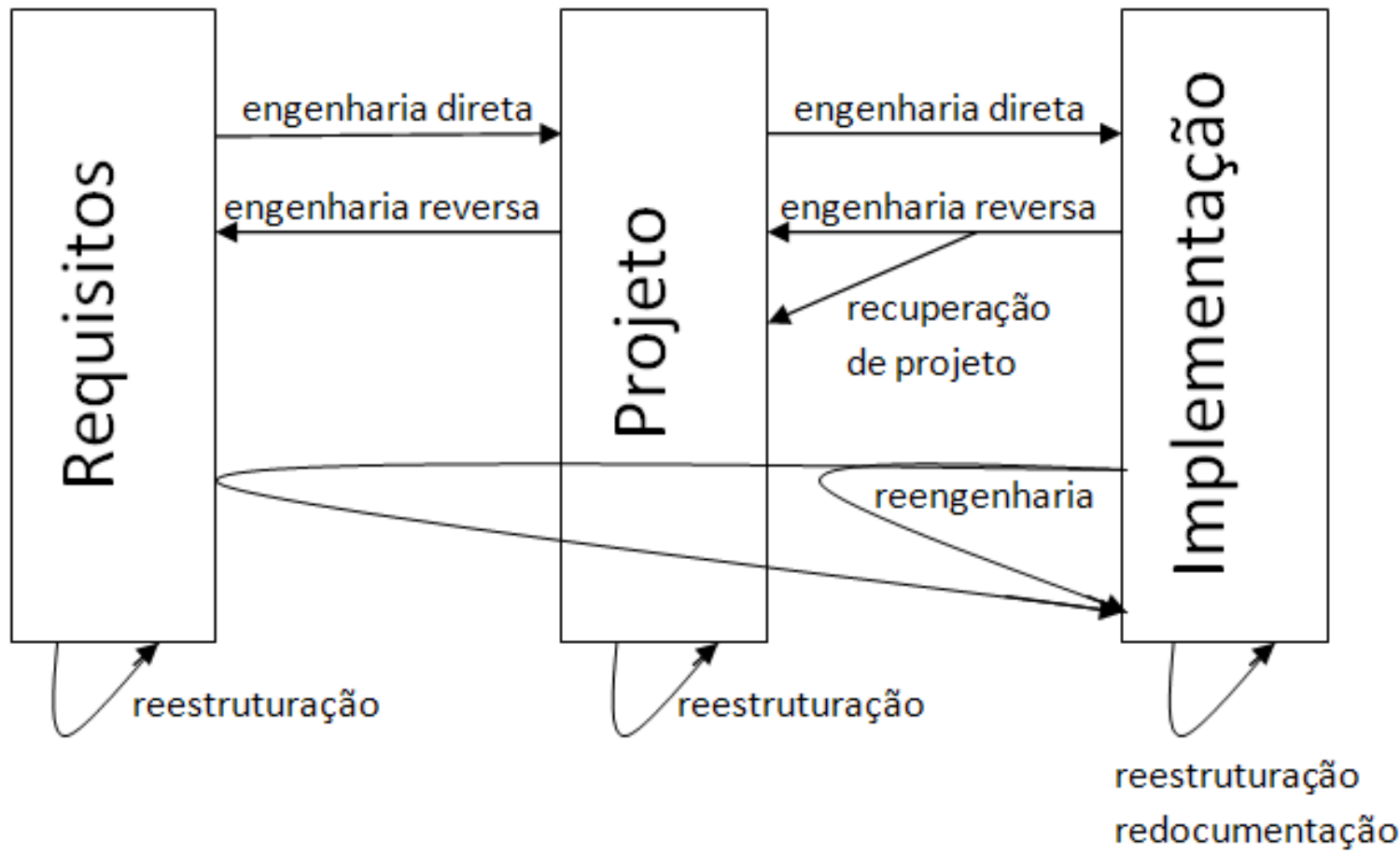


Figura 14-1: Relação esquemática entre os diferentes termos relacionados à engenharia reversa²⁰¹.

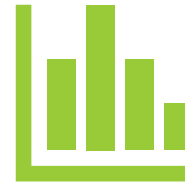
ENGENHARIA REVERSA



De código

Fonte

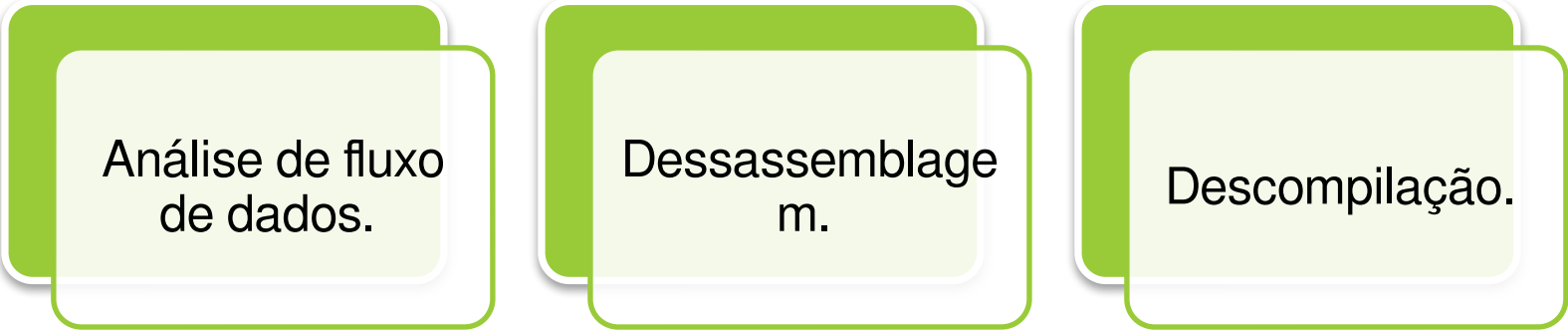
Objeto



De dados



TÉCNICAS DE ENGENHARIA REVERSA DE CÓDIGO



Análise de fluxo de dados.

Dessassemblagem.

Descompilação.

COMPONENTES DE UM DESCOMPILADOR



CARREGADOR.



DESASSEMBLADOR.



IDENTIFICADOR
DE EXPRESSÕES
IDIOMÁTICAS.



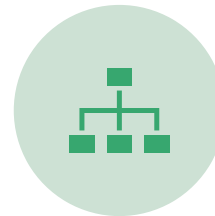
ANÁLISE DE
PROGRAMA.



ANÁLISE DE
FLUXO DE
DADOS.



ANÁLISE DE
TIPOS.



ESTRUTURAÇÃO.



GERAÇÃO DE
CÓDIGO.



ENGENHARIA REVERSA DE DADOS

